

SupportGraph AI

Evidence-Grounded, Safety-Gated Apple Customer Support Resolution System

Technical Project Report

01 — PROJECT OVERVIEW

SupportGraph AI is a customer-support resolution system built from real AppleSupport conversations. The core engineering problem is not simply generating a helpful sentence; it is determining whether a proposed answer is supported by historical evidence and safe to automate.

The implementation separates problem understanding, intent classification, evidence retrieval, evidence validation, response generation, response verification, safety decisions, multi-turn state and human review. This creates an auditable chain between the customer's problem and the final resolution path.

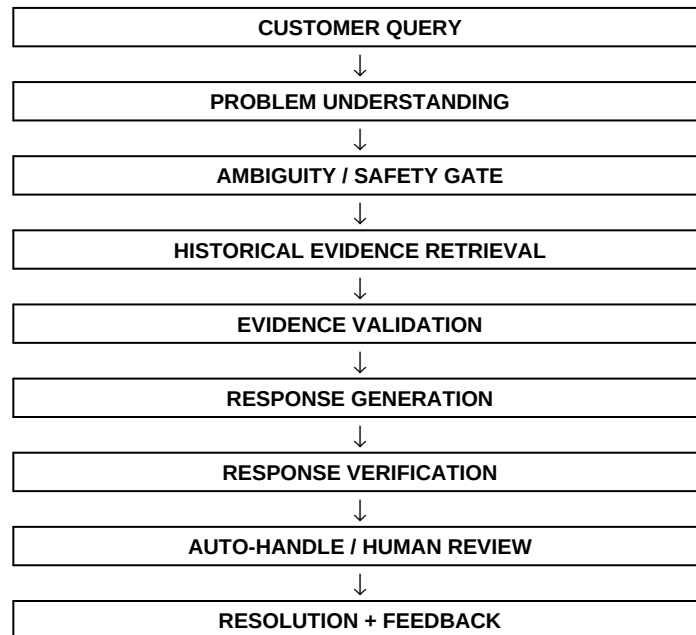


Figure 1: End-to-End Project Technical Flow

Key design principle: the LLM proposes a response; evidence, verification and safety controls decide whether automation is allowed.

02 — PROBLEM STATEMENT

Support conversations are noisy: one message can contain several symptoms, missing context, misleading keywords or a worsening safety condition. A retrieval result can look similar while solving a different problem. A fluent LLM answer can therefore be wrong even when it sounds convincing.

SupportGraph AI was built to make those failure modes explicit and controllable instead of hiding them behind a single generation step.

03 — WHAT WAS IMPLEMENTED

The system is implemented as cooperating components, with each component responsible for a specific decision or control. This makes the resolution path inspectable and testable.

- **Problem understanding:** ProblemExtractor, PrimaryProblemSelector, TurnClassifier and ClarificationEngine convert free-form support language into an operational problem representation.
- **Ambiguity and safety:** AmbiguityAnalyzer and AmbiguityDecisionGate determine whether the system has enough information to proceed. Safety logic can veto normal troubleshooting when critical hardware or thermal signals appear.
- **Retrieval and evidence:** CaseRetriever, HistoricalCorpusIndex and EvidenceRanker retrieve historical precedents. MultiCaseEvidenceSynthesizer, EvidenceConflictDetector and EvidenceValidator determine whether those precedents actually support the current case.
- **Response control:** ResponseGenerator drafts the answer. ResponseVerifier checks symptom alignment, unsupported claims, cause decoupling and evidence corroboration. ResolutionAuditor records the resulting decision.
- **Conversation control:** ConversationState, ConversationManager, ActionTracker and ResolutionProgressEngine preserve context, distinguish confirmed from inferred facts and prevent repeated troubleshooting.
- **Governance and observability:** human-feedback stores, controlled promotion, DecisionTrace, DecisionTraceStore and SystemMetricsAuditor provide evidence governance and operational auditability.

04 — ARCHITECTURE

The pipeline moves sequentially through eight control stages: understanding, classification, the ambiguity/safety gate, retrieval, evidence validation, response generation, response verification and the final automation decision. Each stage is implemented as an independent, inspectable component rather than a single end-to-end model call, so the system's behavior at any point in the chain can be examined on its own terms.

The important architectural boundary is between retrieval and evidence validation. Retrieved cases are candidates for evidence, not evidence by default. The response is then verified after generation before an automated outcome can be returned.

Because each checkpoint is separate, a failure can be traced back to the specific stage that caused it instead of being hidden inside a single opaque generation step. This also means one stage can be corrected or extended without redesigning the rest of the pipeline: the retrieval index can be expanded, or the evidence validator's thresholds can be tightened, without changing how responses are generated or how the safety gate evaluates hardware risk. This modularity is what later allowed real failures to be converted into targeted, stage-specific engineering controls rather than broad, unverifiable fixes.

05 — DATASET AND EVALUATION INTEGRITY

The selected brand is AppleSupport. Brand selection identified 106,719 usable interactions with a selection score of 0.8645, response coverage of 99.93%, reconstructability of 100%, completeness of 0.9997 and diversity of 0.9575.

The final historical retrieval index contains 80,487 non-golden support conversations. A separate frozen set of 200 golden records is reserved for evaluation and excluded from retrieval. This separation protects the benchmark from retrieval leakage.

06 — INTENT UNDERSTANDING AND RETRIEVAL

The final implementation uses 20 operational problem families to make support issues retrievable and comparable. The system combines problem understanding with contextual signals instead of relying only on surface word overlap.

Retrieval is deliberately conservative. Lexical decoy penalties reduce cases that share words but represent a different support problem. Multiple compatible precedents can be synthesized, but synthesis remains downstream of evidence validation.

Important distinction: semantic similarity is a retrieval signal; it is not proof that a historical resolution applies to the current customer.

07 — EVIDENCE VALIDATION

Validation compares each retrieved case against the current conversation across several dimensions, including device, operating system or service, primary symptom, underlying cause and customer intent. A case that matches on surface wording but diverges on device or symptom is treated as weak or insufficient evidence rather than being accepted on similarity alone.

Evidence is assessed as `STRONG_EVIDENCE`, `MODERATE_EVIDENCE`, `WEAK_EVIDENCE`, `CONFLICTING_EVIDENCE` or `INSUFFICIENT_EVIDENCE`. The verdict determines whether the case can support an automated response, requires corroboration or must be escalated.

When multiple retrieved cases point to different causes or resolutions for what looks like the same problem, the evidence is marked as `CONFLICTING_EVIDENCE` and cannot support an automated response on its own. This distinction between similarity and applicability is what allows the system to separate a resolution that merely looks relevant from one that is actually supported by precedent.

08 — RESPONSE GENERATION AND VERIFICATION

Only validated evidence is passed into response construction. The generated response is then checked for symptom alignment, unsupported claims, cause decoupling and corroboration with the retrieved evidence.

A failed verification becomes a verification veto and prevents unsafe auto-handling. This is a key difference from a direct LLM chatbot where generation itself may be treated as the final answer.

09 — SAFETY-GATED AUTOMATION AND HUMAN REVIEW

Automation is allowed only when the problem is sufficiently understood, usable evidence exists, the response passes verification, and no ambiguity or safety condition requires specialist intervention.

Escalation decisions include `HUMAN_REQUIRED`, `INSUFFICIENT_INFORMATION`, `VERIFICATION_VETO`, `EVIDENCE_LIMITED`, `GENUINE_AMBIGUITY`, `MULTI_PROBLEM_COMPLEXITY` and `RECOVERABLE_ESCALATION`.

If any required gate fails, the path moves to human review rather than forcing an automated resolution.

Safety-critical example: a customer reports rapidly draining battery followed by extreme heat, smoke and cracking back glass. The worsening thermal/hardware signals trigger urgent human escalation rather than continuing ordinary troubleshooting.

A second adversarial case combined a duplicate billing charge with Apple ID lockout. Because the system could not safely verify an appropriate account or refund action, it escalated instead of inventing a financial workflow.

10 — MULTI-TURN RESOLUTION

The conversation layer maintains confirmed facts separately from inferred facts, tracks canonical troubleshooting actions, recognizes semantic aliases of previously attempted steps, and limits clarification to one decision-critical question per turn.

Verified context retention was 100% and repeat prevention was 100%. Original adversarial scenarios achieved 30/30 passes; the unseen adversarial set achieved 20/20 passes.

11 — HUMAN FEEDBACK AND EVIDENCE GOVERNANCE

Human feedback is not written directly into live evidence. It enters a controlled lifecycle so that specialist corrections can be reviewed, evaluated and traced before influencing approved evidence.

Candidate and approved evidence stores remain separate. Provenance, versioning, offline benchmark simulation and leakage checks protect the system from an unsafe feedback loop. There is no automatic online promotion.

12 — OBSERVABILITY AND AUDITABILITY

The system records a 12-step decision trace covering the important stages of the resolution path. Runtime monitoring includes latency, health, alerts, evidence provenance, feedback monitoring, auditability and PII/secret redaction.

Because every gate decision is written to the trace, a specialist reviewing an escalated or automated case can reconstruct exactly why the system classified the intent the way it did, which evidence it accepted or rejected, and which check ultimately allowed or blocked automation. This turns observability from a passive log into an active audit trail that supports both individual case review and system-wide quality checks.

The final observability suite reported 33/33 tests passing. The broader implementation was also validated through regression, adversarial and evaluation gates.

13 — REAL FAILURES CONVERTED INTO ENGINEERING CONTROLS

- **AirPods pairing recognition gap:** the system initially missed some pairing expressions. The mapping was expanded to cover AirPods, EarPods, headphones, Bluetooth and pairing terminology, followed by regression testing.
- **Thermal hazard detection gap:** worsening-device language was expanded to include smoking, burning, swelling, extreme heat and fire-related signals. A global hardware/thermal safety veto was added.
- **Billing + account-lockout decoy:** verification correctly blocked an unsafe financial/account workflow and escalated the case.
- **Retrieval coverage gap:** retrieval coverage was expanded through the 20 operational problem families and the 80,487-case historical index.

The important point is that failures became new controls and regression/adversarial tests, not merely manual exceptions.

14 — EVALUATION RESULTS

The final evaluation measures both model/system capability and the safety of the automation boundary.

Intent and problem understanding: primary intent accuracy 88.3%, top-2 intent accuracy 96.1%, and problem family accuracy 94.8%.

Evidence quality: usable evidence coverage 89.0% and direct problem match 84.0%.

Automation safety: safe auto-handle rate 60.5%, auto-handle precision 100%, unsafe auto-handles 0.

Integrity: evaluation leakage 0; context retention 100%; repeat prevention 100%.

Adversarial validation: 90.0% overall pass rate, with 30/30 original and 20/20 unseen adversarial scenarios passing.

Latency: P50 80.7 ms, P95 414.2 ms, P99 480.8 ms.

Additional verification recorded 100% safe escalation of verification failures and complete handling of tested conflict and ambiguity cases.

15 — WHY THIS SYSTEM IS DIFFERENT

The uniqueness is the control architecture around the language model. SupportGraph AI does not equate retrieval with evidence, generation with correctness, or confidence with permission to automate. It creates separate gates for understanding, evidence quality, response grounding, safety, conversation state and human governance.

In a conventional support chatbot, a single generation step is often treated as the final answer: if the response sounds fluent and confident, it is returned to the customer regardless of whether it is grounded in anything verifiable. SupportGraph AI instead treats generation as only one stage in a longer decision process, where a fluent-sounding response can still be rejected if it fails evidence validation or response verification.

This distinction matters most in edge cases: a worsening thermal condition, a billing dispute tied to an account lockout, or retrieved precedents that disagree with each other. A direct-generation system has no built-in mechanism to recognize that it should stop and defer to a human in these situations, whereas SupportGraph AI's gates are specifically designed to catch these conditions before an automated response is ever sent.

16 — FINAL TECHNICAL CONTRIBUTION

SupportGraph AI connects a customer's problem to historical support evidence, validates that evidence against the current case, generates a proposed resolution, verifies the response, and then makes a governed decision between automated handling and human escalation.

Customer problem → structured understanding → historical evidence → validated resolution → verified response → safe automation or human escalation.

The result is a support system that can be inspected, tested and improved through evidence rather than relying on fluent generation alone.

Implementation footprint: 80,487 indexed historical cases · 20 operational problem families · multi-turn state tracking · evidence validation · response verification · safety escalation · governed human feedback · 12-step decision tracing.